```
        duty = 8'd220; #200;

        $finish;
    end
endmodule
```

Waveform reading tip: Zoom out and you'll see duty cycle differences as "density" of high pulses.

Mini-build 3: UART TX "hello" block (hardware that talks)

UART is the rite of passage because:

- it introduces timing derived from clocks
- it produces observable serial behavior
- it's the foundation of debugging on FPGA boards later

We'll implement a basic UART transmitter:

- 1 start bit (0)
- 8 data bits (LSB first)
- 1 stop bit (1)

UART TX RTL: uart_tx.v

```
module uart_tx #(
    parameter CLK_HZ   = 50_000_000,
    parameter BAUD     = 115200
)(
   input        clk,
    input        rst_n,
    input        send,
    input  [7:0] data,
    output reg   tx,
    output reg   busy
);
   localparam integer DIV = CLK_HZ / BAUD;

    reg [15:0] div_ctr;
    reg [3:0]  bit_idx;
    reg [9:0]  shifter; // start + 8 data
+ stop

    always @(posedge clk) begin
        if (!rst_n) begin
            tx      <= 1'b1;
            busy    <= 1'b0;
            div_ctr <= 0;
            bit_idx <= 0;
            shifter <= 10'h3FF;
        end else begin
            if (!busy) begin
                if (send) begin
                    // Load frame: {stop,
data[7:0], start}
                        shifter <= {1'b1,
data, 1'b0};
                    busy    <= 1'b1;
                    div_ctr <= 0;
                    bit_idx <= 0;
                end
            end else begin
              if (div_ctr == DIV-1) begin
                    div_ctr <= 0;
                   tx      <= shifter[0];
                       shifter <= {1'b1,
shifter[9:1]}; // shift right, fill with 1s
                    bit_idx <= bit_idx +
1'b1;

                    if (bit_idx == 4'd9)
begin
                       busy <= 1'b0;
                       tx   <= 1'b1;
                    end
                end else begin
                    div_ctr <= div_ctr +
1'b1;
                end
            end
        end
    end
endmodule
```

UART TX testbench: tb_uart_tx.v

We'll simulate at a smaller clock to keep waveform readable.

```
`timescale 1ns/1ps

module tb_uart_tx;
    reg clk, rst_n;
    reg send;
    reg [7:0] data;
    wire tx;
    wire busy;

      // Use a small clock + baud for
sim clarity
         uart_tx  #(.CLK_HZ(1_000_000),
.BAUD(10_000)) dut (
       .clk(clk), .rst_n(rst_n), .send(send),
.data(data), .tx(tx), .busy(busy)
    );

    initial clk = 0;
    always #500 clk = ~clk; // 1 MHz clock
=> period 1000ns
```